A Comparative Study of Nonlinear MPC and Differential-Flatness-Based Control for Quadrotor Agile Flight

Sihao Sun, Angel Romero, Philipp Foehn, Elia Kaufmann and Davide Scaramuzza

arXiv:2109.01365v3 [cs.RO] 23 Feb 2022

***Abstract*—Accurate trajectory tracking control for quadrotors is essential for safe navigation in cluttered environments. However, this is challenging in agile flights due to nonlinear dynamics, complex aerodynamic effects, and actuation constraints. In this article, we empirically compare two state-of-the-art control frameworks: the nonlinear-model-predictive controller (NMPC) and the differential-flatness-based controller (DFBC), by tracking a wide variety of agile trajectories at speeds up to 20 $\mathrm{m/s}$ (i.e., 72 $\mathrm{km/h}$). The comparisons are performed in both simulation and real-world environments to systematically evaluate both methods from the aspect of tracking accuracy, robustness, and computational efficiency. We show the superiority of NMPC in tracking dynamically infeasible trajectories, at the cost of higher computation time and risk of numerical convergence issues. For both methods, we also quantitatively study the effect of adding an inner-loop controller using the incremental nonlinear dynamic inversion (INDI) method, and the effect of adding an aerodynamic drag model. Our real-world experiments, performed in one of the world's largest motion capture systems, demonstrate more than 78% tracking error reduction of both NMPC and DFBC, indicating the necessity of using an inner-loop controller and aerodynamic drag model for agile trajectory tracking.**

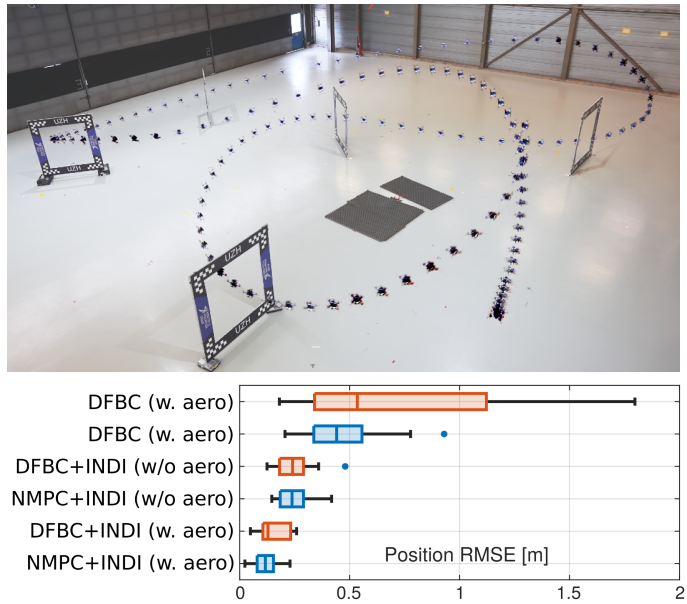


Fig. 1: **Top:** One quadrotor tracking a race trajectory. **Bottom:** Box plot comparing the position tracking root-mean-square-error (RMSE) of NMPC, DFBC, and their variations with INDI inner-loop, and whether the aerodynamic drag effect is considered. For each method, data are collected from real-world flights tracking different reference trajectories with speeds up to **20** $\mathrm{m/s}$ and accelerations up to **5**g.

I. Introduction

A. Motivation

Quadrotors are extremely agile. Exploiting their agility is crucial for time-critical missions, such as search and rescue, aerial delivery, drone racing, and even flying cars [1, 2]. An accurate trajectory tracking controller is required to safely execute high-speed trajectories in cluttered environments. However, most approaches struggle to handle joint effects in agile flights, such as nonlinear dynamics, aerodynamic effects, and actuation limits.

Recently, nonlinear model predictive control (NMPC) has drawn much attention for quadrotor control thanks to advances in hardware and algorithmic efficiency [3–10]. NMPC particularly excels in handling control limits, and its predictive nature is believed to be beneficial for trajectory tracking at high speeds [3]. A recent study has demonstrated its performance in tracking aggressive trajectories up to 20 $\mathrm{m/s}$ [4].

However, NMPC is computationally extremely demanding compared to the state-of-the-art non-predictive method: the differential-flatness-based controller (DFBC) [11, 12]. This method also shows great potential in accurately tracking agile trajectories autonomously. A recent study has used DFBC to track trajectories up to 12.9 $\mathrm{m/s}$ with 2.1 g accelerations, with only 6.6 centimeters position tracking error [12]. Although the state-of-the-art DFBC has not achieved agile flights as fast as NMPC showed in [4], its high computational efficiency and tracking accuracy render the necessity of applying NMPC for agile trajectory tracking questionable. Therefore, a comparative study of NMPC and DFBC is needed to provide insights for future research on fully autonomous agile flights, in order to further improve their efficiency and reliability.

B. Contribution

This article presents the first comparative study of the two state-of-the-art tracking controllers: NMPC and DFBC method, in fast trajectory tracking with speed up to 20 $\mathrm{m/s}$ (i.e., 72 $\mathrm{km/h}$) and 5 g accelerations. Specifically, we select the NMPC method that uses the full nonlinear dynamics

All authors are with the Robotics and Perception Group, University of Zurich, Switzerland (http://rpg.ifi.uzh.ch). This work was supported by the National Centre of Competence in Research (NCCR) Robotics through the Swiss National Science Foundation (SNSF) and the European Union's Horizon 2020 Research and Innovation Programme under grant agreement No. 871479 (AERIAL-CORE) and the European Research Council (ERC) under grant agreement No. 864042 (AGILEFLIGHT).

with proper actuator bounds and regards single rotor thrusts as control inputs. This NMPC has been applied in previous works, such as [3, 5]. For a fair comparison, we improve the DFBC method used in [11, 12] with a control allocation approach using constrained quadratic programming [13] also to consider control input limits.

All experiments are conducted in both simulations and the real world. The simulations compare in both ideal and practical conditions with model-mismatch, estimation latency, and external disturbances. The real-world experiments are conducted in one of the world's largest motion capture systems, with $30\times30\times8$ m^3 flight volume. Multiple agile trajectories are selected as reference, including not only dynamically *feasible* trajectories, but also dynamically *infeasible* trajectories that require thrusts exceeding the maximum capacity of the quadrotor motors, which is likely to happen in high-speed flights due to model mismatch. These tests investigate the performance of both methods in the presence of significant high-speed-induced aerodynamic effects, inevitable system latency, and on an onboard embedded computer.

The experimental results reveal that NMPC is significantly more computationally demanding, and more prone to suffer from numerical convergence issues in the presence of large external force disturbances. However, NMPC also excels in tracking dynamically *infeasible* trajectories, making it more suitable for tracking time-optimal trajectories that violate the rotor thrust constraints.

In addition, this study also highlights the importance of implementing an inner-loop controller for robustification. Specifically, we select the incremental nonlinear dynamic inversion (INDI) method as the inner-loop angular controller, thanks to its simplicity in implementation and demonstrated robustness in various real-world experiments [14, 15], including a combination with DFBC [12]. As for NMPC, differently from the state-of-the-art using a PID as the low-level control [4], we propose a method to hybridize NMPC with INDI that considers the real input limits of the quadrotor instead of constraints on virtual inputs. Real-world flight results demonstrate more than 78% position tracking error reduction of NMPC and DFBC with an INDI inner-loop. We also reveal that a well-selected inner-loop controller is significantly more important than simply considering the aerodynamic effects, as is compared in Fig. 1.

Apart from the technical contribution, this paper can also be regarded as a tutorial for non-expert readers on agile quadrotor flight. We encourage the practitioners to use the presented results as a baseline for further development of both DFBC and NMPC approaches.

II. Related Work

In this paper, we classify the trajectory tracking controller into two categories: non-predictive and predictive methods. While the predictive methods encode multiple future time-steps into the control command, the non-predictive methods only track a single reference step. In the following, we review works towards improving quadrotor trajectory tracking accuracy from these two different methodologies. A more comprehensive survey of quadrotor position and attitude control methods can be found in the literature (e.g., see [16, 17]).

A. Non-predictive Quadrotor Trajectory Tracking Control

Unlike most fixed-wing aircraft, quadrotors are inherently unstable. Therefore, the initial work of quadrotor control aimed at achieving stable hovering and near-hover flights. Thanks to the small-angle assumptions in these conditions, linear control methods such as PID and LQR demonstrate sufficiently good performance (see, e.g., [18, 19]).

However, as the requirements for agile flight emerges, these assumptions are no longer valid. Nonlinearities from the attitude dynamics are the first problem to tackle. For this reason, nonlinear flight controllers are proposed, such as feedback linearization [20] and backstepping [21]. In order to cope with the singularities of Euler angles as the nonlinear attitude representation, quaternions are widely adopted to parametrize the attitude [22]. In addition, the authors of [23] propose the geometric tracking controller to directly control the quadrotor on the manifold of the special Euclidean group SE(3), showing almost globally asymptotic tracking of the position, velocity, and attitude of the quadrotor.

A seminal work [24] reveals that quadrotors are differentially flat systems. By virtue of this property, given the time-parameterized 3D path, one can derive the reference attitude, angular rate, and accelerations. These references can be sent to a closed-loop flight controller as feedforward terms, while additional feedback control is required to address model mismatch and external disturbances. As such, differential-flatness based controller (DFBC) has significantly improved the tracking performance at relatively high speeds [11, 12].

As the flying speed increases, a quadrotor starts experiencing non-negligible aerodynamic effects, including drag [25], aerodynamic torque [26], and variation of thrusts [27]. Authors of [11] show that the aerodynamic drag does not affect the differential flatness of a quadrotor. Thus they adopt a first order aerodynamic model with feedforward terms derived from the reference trajectory to improve the tracking performance. Since an accelerometer can directly read external forces, [12] leverages accelerometer measurements to improve the tracking accuracy instead of resorting to an aerodynamic drag model. This method also demonstrates remarkable performance in disturbance rejection and platform adaptability.

Effectively handling control input limits is a remaining challenge for non-predictive methods, including DFBC. So far, existing methods have prioritized the position tracking over heading using various approaches, such as redistributed pseudo inversion [28], weighted-least-square allocation [29], control-prioritization method [30, 12], and constrained-quadratic-programming allocation [31]. While these methods can mitigate the actuator saturation effect when the trajectories are dynamically *feasible*, its performance in tracking dynamically *infeasible* trajectories is still questionable. In this work, we will push the limit of DFBC to conduct agile trajectory tracking tasks to a much higher flight speed and acceleration than the state-of-the-art, and study its performance in tracking dynamically *infeasible* trajectories where violating rotor thrust limits becomes inevitable.

B. Model Predictive Control for Quadrotor Trajectory Tracking

Model predictive control (MPC) is a prevalent method in robots thanks to its predictive nature and ability to handle input constraints[10, 32]. It generates control commands in receding horizon fashion, which minimizes the tracking error in the predicted time horizon by solving constrained optimization problems.

However, MPC is very computationally demanding compared with non-predictive methods. Especially, using nonlinear-MPC (NMPC) with a full-state nonlinear model of a quadrotor was computationally intractable onboard early-age unmanned-aerial vehicles (UAVs). For this reason, linear-MPC (LMPC) is adopted in many studies for only position control [33], or control linearized model based on small-angle assumptions [34]. Therefore, these LMPC methods cannot fully capture nonlinearities in rotational dynamics, and underperforms NMPC methods [10, 32]

Separating flight controllers into high-level position control and low-level attitude control is another common approach to simplify the model and alleviate the computational load of NMPC [6, 7, 9]. However, in such a cascaded control architecture, the high-level controller cannot precisely capture the real quadrotor capability because they often ignore the effects of system limitation (such as [6]), or introduce a virtual constraint on states [9]. Consequently, the command fed into the lower-level controller is either too conservative to achieve agile flights, or over-aggressive that causes instability.

Thanks to the recent development in hardware and nonlinear optimization solvers [35–37], running NMPC with a nonlinear full dynamic model becomes computationally tractable on an embedded computer. Hence, recently, some studies start using the full-nonlinear dynamic model, and regarding single rotor thrusts as inputs for NMPC, thus are able to fully exploit quadrotors capability [3–5]. These methods either directly use the optimized single rotor thrust commands [3], or send intermediate states from the solution (such as the angular rates) to a low-level controller [4, 5]. A recent study [4] demonstrates the ability of the full-model NMPC with a PID low-level controller in tracking a pre-planned race trajectory at speed up 20 $\mathrm{m/s}$, which surpasses the fastest speed (12.9 $\mathrm{m/s}$ in [12]) achieved using DFBC, but also with a much larger tracking error (0.7 m v.s. 0.066 m).

Although running NMPC is realizable on modern embedded computers, it still requires significantly more computational resources than non-predictive methods represented by DFBC. For this reason, NMPC may suffer from numerical convergence issues, especially when the platform lacks a sufficient computational budget. Moreover, the advantage of NMPC becomes questionable as DFBC can also address input limits using the control allocation technique and generate feedforward control leveraging differential-flatness property. Therefore, it is necessary to compare these two methodologies and understand at what conditions each approach is preferable to provide insights and recommendations for future applications.

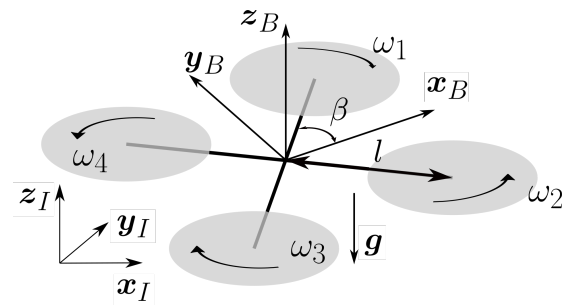


Fig. 2: Coordinate definitions and propeller numbering convention.

III. Preliminaries

A. Notations

Throughout the paper, we use subscription r to indicate the reference variables derived from the reference trajectory. Subscription d indicates the desired value, which is calculated from a higher-level controller. We use bold lowercase letters to represent vectors and bold uppercase letters for matrices; otherwise, they are scalars. Two right-handed coordinate frames are used in this paper: they are the inertial-frame $\mathcal{F}_{\mathcal{I}}$: $\{\boldsymbol{x}_I, \boldsymbol{y}_I, \boldsymbol{z}_I\}$ with $\boldsymbol{z}_I$ pointing upward opposite to the gravity, and the body-frame $\mathcal{F}_{\mathcal{B}}$: $\{\boldsymbol{x}_B, \boldsymbol{y}_B, \boldsymbol{z}_B\}$ with $\boldsymbol{x}_B$ pointing forward and $\boldsymbol{z}_B$ aligned with the collective thrust direction (see Fig. 2). Vectors with superscription B are expressed in the body-frame; otherwise they are expressed in the inertial-frame. The rotation from $\mathcal{F}_I$ to $\mathcal{F}_B$ is represented by rotational matrix $\boldsymbol{R}(\boldsymbol{q}) = [\boldsymbol{x}_B, \boldsymbol{y}_B, \boldsymbol{z}_B] \in \mathrm{SO}(3)$ parameterized by quaternion $\boldsymbol{q} = [q_w,\ q_x,\ q_y,\ q_z]^T \in \mathbb{S}^3$. We use subscript $\{x, y, z\}$ to represent the imaginary components of a quaternion, namely $\boldsymbol{q}_{x,y,z} = [q_x,\ q_y,\ q_z]^T$. Operator $\mathrm{diag}(A_1, A_2, ..., A_n)$ denotes a diagonal matrix with scalars or matrices $(A_1, A_2, ..., A_n)$ as diagonal entries.

B. Quadrotor Model

1) Quadrotor Rigid-Body Model: The quadrotor model is established using 6-DoF rigid body kinematic and dynamic equations. For translational dynamics, we have

$$\ddot{\boldsymbol{\xi}} = (T\boldsymbol{z}_B + \boldsymbol{f}_a)/m + \boldsymbol{g}, \tag{1}$$

where $\boldsymbol{\xi}$ denotes the position of the quadrotor center of gravity (CoG); T and m are the collective thrust and total mass respectively; $\boldsymbol{g} \in$ is the gravitational vector; $\boldsymbol{f}_a$ indicates the exogenous aerodynamic drag force during high-speed flights.

The rotational kinematic and dynamic equations are expressed as

$$\dot{\boldsymbol{q}} = \frac{1}{2}\boldsymbol{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\Omega}^B \end{bmatrix}, \tag{2}$$

$$\boldsymbol{I}_v\dot{\boldsymbol{\Omega}}^B = \boldsymbol{I}_v\boldsymbol{\alpha}^B = -\boldsymbol{\Omega}^B \times \boldsymbol{I}_v\boldsymbol{\Omega}^B + \boldsymbol{\tau} + \boldsymbol{d}_\tau, \tag{3}$$

where $\otimes$ denotes the quaternion multiplication operator. $\boldsymbol{\Omega}$ is the angular velocity of $\mathcal{F}_B$ with respect to $\mathcal{F}_I$. Its derivative, namely angular acceleration, is denoted by $\boldsymbol{\alpha}$. Throughout the paper, we use its projection on $\mathcal{F}_B$, namely $\boldsymbol{\Omega}^B = [\Omega_x,\ \Omega_y,\ \Omega_z]^T$, since its directly measurable from the inertial measurement unit (IMU). $\boldsymbol{I}_v$ indicates the inertia matrix of the

entire quadrotor. $\boldsymbol{\tau}$ is the resultant torque generated by rotors. $\boldsymbol{d}_\tau$ is the model uncertainties on the body torque, which can come from high-order aerodynamic effects, center of gravity bias, or even minor distinctions among the rotors.

The collective thrust and rotor generated torques are functions of rotor speeds $\boldsymbol{\omega}$:

$$\begin{bmatrix} T \\ \boldsymbol{\tau} \end{bmatrix} = \boldsymbol{G}_1\boldsymbol{u} + \boldsymbol{G}_2\dot{\boldsymbol{\omega}} + \boldsymbol{G}_3(\boldsymbol{\Omega})\boldsymbol{\omega} \tag{4}$$

where

$$\boldsymbol{u} = c_t\boldsymbol{\omega}^{\circ 2} \tag{5}$$

represents the thrust generated by each rotor and $^\circ$ indicates the Hadamard power. c_t is the thrust coefficient. $\boldsymbol{\omega}$ is the vector of angular rates of each propeller. $\boldsymbol{G}_1$ to $\boldsymbol{G}_3$ are matrices defined as

$$\boldsymbol{G_1} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ l\sin\beta & -l\sin\beta & -l\sin\beta & l\sin\beta \\ -l\cos\beta & -l\cos\beta & l\cos\beta & l\cos\beta \\ c_q/c_t & -c_q/c_t & c_q/c_t & -c_q/c_t \end{bmatrix} \tag{6}$$

$$\boldsymbol{G_2} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ I_p & -I_p & I_p & -I_p \end{bmatrix} \tag{7}$$

$$\boldsymbol{G_3} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ I_p\Omega_y & -I_p\Omega_y & I_p\Omega_y & -I_p\Omega_y \\ -I_p\Omega_x & I_p\Omega_x & -I_p\Omega_x & I_p\Omega_x \\ 0 & 0 & 0 & 0 \end{bmatrix} \tag{8}$$

where c_q is the torque coefficient; β and l are geometric parameters defined as per Fig. 2. I_p is the inertia of the rotor around $\boldsymbol{z}_B$.

Terms $\boldsymbol{G}_2\dot{\boldsymbol{\omega}}$ and $\boldsymbol{G}_3(\boldsymbol{\Omega})\boldsymbol{\omega}$ are torques respectively due to angular acceleration of rotors and gyroscopic effects, which are usually neglected for controller design. However, in Sec IV-C, we will revisit the INDI method [12] for angular acceleration control that takes into account effects of the inertial torque term $\boldsymbol{G}_2\dot{\boldsymbol{\omega}}$.

2) Aerodynamic Drag Model: Quadrotors during high-speed flight experience significant aerodynamic drag forces, which need to be precisely modeled to improve tracking accuracy while minimizing the computational overhead. In this work, we use an aerodynamic drag model which captures the major effects, and is proved effective in works such as [11].

$$\boldsymbol{f}_a^B = \begin{bmatrix} -k_{d,x}v_x \\ -k_{d,y}v_y \\ -k_{d,z}v_z + k_h\left(v_x^2 + v_y^2\right) \end{bmatrix} \tag{9}$$

where $[v_x,\ v_y,\ v_z] = \boldsymbol{R}(\boldsymbol{q})^T\dot{\boldsymbol{\xi}}$ (i.e., the projection of velocity in the body frame; here we assume zero wind-speed). $k_{d,x,y,z}$ and k_h are positive parameters can be identified from flight data.

IV. Methodologies

This section elaborates the two control methods compared. A nonlinear NMPC method is selected that considers the thrust limits of each rotor, the full nonlinear dynamics, and the aerodynamic effects. As for the DFBC method, we improve the state-of-the-art such that these factors are also addressed, which ensures a fair comparison with NMPC. Finally, both methods are augmented with an INDI controller [14] to convert the single rotor thrust commands to rotor-speed commands, while improving the robustness against model uncertainties and disturbances on the rotational dynamics. The control diagrams of both methods are illustrated in Fig. 4 and Fig. 3.

A. Nonlinear Model Predictive Controller

NMPC generates control commands by solving a finite-time optimal control problem (OCP) in a receding horizon fashion. Given a reference trajectory, the cost function is the error between the predicted states and the reference states in the time horizon, meaning that multiple reference points in the time horizon are used. In order to conduct numerical optimizations, we discretize the states and inputs into N equal intervals over the time horizon $\tau \in [t,\ t+h]$ of size $dt = h/N$ with h denoting the horizon length, yielding a constrained nonlinear optimization problem:

$$\begin{aligned} \boldsymbol{u}_{\text{NMPC}} = \underset{\boldsymbol{u}}{\operatorname{argmin}} \sum_{k=0}^{N-1} &\left(||\boldsymbol{x}_k - \boldsymbol{x}_{k,r}||_{\boldsymbol{Q}} + ||\boldsymbol{u}_k - \boldsymbol{u}_{k,r}||_{\boldsymbol{Q}_u}\right) \\ &+ ||\boldsymbol{x}_N - \boldsymbol{x}_{N,r}||_{\boldsymbol{Q}_N} \\ s.t. \quad \boldsymbol{x}_{k+1} &= \boldsymbol{f}(\boldsymbol{x}_k, \boldsymbol{u}_k), \qquad \boldsymbol{x}_0 = \boldsymbol{x}_{\text{init}}, \\ \boldsymbol{\Omega}^B &\in \left[\boldsymbol{\Omega}^B_{\min}\ \boldsymbol{\Omega}^B_{\max}\right], \quad \boldsymbol{u} \in [\boldsymbol{u}_{\min}\ \boldsymbol{u}_{\max}], \end{aligned} \tag{10}$$

Note that, we use the thrust of rotors $\boldsymbol{u}$ defined in (4) and (5) as the control input of NMPC. The state vector is defined as $\boldsymbol{x} = \left[\boldsymbol{\xi}\ \dot{\boldsymbol{\xi}}\ \boldsymbol{q}\ \boldsymbol{\Omega}^B\right]$, and

$$\boldsymbol{Q} = \operatorname{diag}\left(\boldsymbol{Q}_\xi,\ \boldsymbol{Q}_v,\ \boldsymbol{Q}_q,\ \boldsymbol{Q}_\Omega\right),\ \boldsymbol{Q}_N = \boldsymbol{Q} \tag{11}$$

The reference state vector $\boldsymbol{x}_r$ and input $\boldsymbol{u}_r$ can be obtained from a trajectory planner which generates full states such as the one introduced in [4]. Function $\boldsymbol{f}\left(\boldsymbol{x}_k, \boldsymbol{u}_k\right)$ is the discretized version of nonlinear quadrotor model (1)-(5). The same as many other works (see, e.g., [3, 38]), we omit $\boldsymbol{G}_2$ and $\boldsymbol{G}_3$ related terms in (4) as their effects are negligible. $\boldsymbol{x}_{\text{init}}$ is the current state estimation when solving the OCP. $\boldsymbol{u}_{\min}$ and $\boldsymbol{u}_{\max}$ are minimum and maximum values of motor thrusts. Constraints on angular velocities are also added, which is found beneficial for improving the stability of NMPC according to [5]. Note that the following abuse of notation is used when calculating quaternion error in the above optimization problem:

$$\boldsymbol{q} - \boldsymbol{q}_r = \left(\boldsymbol{q} \otimes \boldsymbol{q}_r^{-1}\right)_{x,y,z} \tag{12}$$

The above NMPC solves the full nonlinear model of a quadrotor, instead of resorting to a cascaded structure, or linear assumptions. In this paper, this quadratic nonlinear optimization problem is solved by a sequential quadratic programming

(SQP) algorithm executed in a real-time iteration scheme [39]. We implement this algorithm using ACADO [40] toolkit with qpOASES [41] as the solver.

B. Differential-Flatness Based Controller

Quadrotors are differentially flat systems [24], namely, all the states and inputs can be written as algebraic functions of the flat outputs and their derivatives. This allows a direct mapping from the flat outputs (positions ξ and heading ψ) to the angular rates $\boldsymbol{\Omega}^B$ and angular accelerations $\boldsymbol{\alpha}_r^B$, which are leveraged by DFBC as feed-forward terms to improve the tracking accuracy.

In the following, we introduced the DFBC method improved from a previous work [24], where the original geometric attitude controller is replaced by the tilt-prioritized method [30]. We also use the quadratic-programming based control allocation [13] to address input constraints. These modifications are beneficial in tracking dynamically *infeasible* trajectories, as will be discussed in Sec. V-E. Fig. 4 presents an overview of this method.

1) Desired Attitude and Collective Thrust: First of all, we calculate the desired acceleration from a PD controller:

$$\ddot{\boldsymbol{\xi}}_d = \boldsymbol{K}_\xi\left(\boldsymbol{\xi}_r - \boldsymbol{\xi}\right) + \boldsymbol{K}_v\left(\dot{\boldsymbol{\xi}}_r - \dot{\boldsymbol{\xi}}\right) + \ddot{\boldsymbol{\xi}}_r \tag{13}$$

where $\boldsymbol{K}_\xi$ and $\boldsymbol{K}_v$ are positive-definite diagonal gain matrices. Then from (1), the desired thrust T_d and thrust direction $\boldsymbol{z}_{B,d}$ are obtained as

$$T_d\boldsymbol{z}_{B,d} = \left(\ddot{\boldsymbol{\xi}}_d - \boldsymbol{g}\right)m - \boldsymbol{f}_a \tag{14}$$

Given reference heading angle ψ_r, we get an intermediate axis $\boldsymbol{x}_{C,d}$, by which the desired attitude can be obtained by the following equations:

$$\boldsymbol{x}_{C,d} = [\cos\psi_r,\ \sin\psi_r,\ 0]^T, \tag{15}$$

$$\boldsymbol{y}_{B,d} = \frac{\boldsymbol{z}_{B,d}\times\boldsymbol{x}_{C,d}}{||\boldsymbol{z}_{B,d}\times\boldsymbol{x}_{C,d}||}, \tag{16}$$

$$\boldsymbol{R}(\boldsymbol{q}_d) = [\boldsymbol{x}_{B,d},\ \boldsymbol{y}_{B,d},\ \boldsymbol{z}_{B,d}]. \tag{17}$$

where $\boldsymbol{q}_d$ is the desired attitude expressed by the quaternion.

2) Reference Angular Velocity and Acceleration: We leverage the differential flatness property of a quadrotor to derive the reference angular velocity and angular accelerations. Using them into the attitude control can help in tracking jerk and snap (3rd and 4th order derivative of position $\boldsymbol{\xi}$), which is found essential for tracking accuracy [12].

Taking the derivative of both sides of (1) and assuming a constant external aerodynamic force $\boldsymbol{f}_a$, we have

$$m\dddot{\boldsymbol{\xi}} = \dot{T}\boldsymbol{z}_B + T\boldsymbol{\Omega}\times\boldsymbol{z}_B. \tag{18}$$

Then given reference jerk $\dddot{\boldsymbol{\xi}}_r$ and substitute it for $\dddot{\boldsymbol{\xi}}$ in (18), we get

$$\boldsymbol{h}_\Omega \triangleq \boldsymbol{\Omega}\times\boldsymbol{z}_B = \frac{m}{T}(\dddot{\boldsymbol{\xi}}_r - \dot{T}\boldsymbol{z}_B), \tag{19}$$

by which the reference angular velocity can be obtained as

$$\boldsymbol{\Omega}_r^B = \begin{bmatrix}-\boldsymbol{h}_\Omega\cdot\boldsymbol{y}_B, & \boldsymbol{h}_\Omega\cdot\boldsymbol{x}_B, & \dot{\psi}_r\boldsymbol{z}_W\cdot\boldsymbol{z}_B\end{bmatrix}^T, \tag{20}$$

where ψ_r is the reference heading angle.

We further take the derivative on both sides of (18) and uses snap reference $\ddddot{\boldsymbol{\xi}}_r$ to replace $\ddddot{\boldsymbol{\xi}}$, yielding

$$\begin{aligned}\boldsymbol{h}_{\boldsymbol{\alpha}} &\triangleq \dot{\boldsymbol{\Omega}}\times\boldsymbol{z}_B \\ &= \frac{m}{T}\ddddot{\boldsymbol{\xi}}_r - \left(\boldsymbol{\Omega}\times(\boldsymbol{\Omega}\times\boldsymbol{z}_B) + \frac{2\dot{T}}{T}\boldsymbol{\Omega}\times\boldsymbol{z}_B + \frac{\ddot{T}}{T}\boldsymbol{z}_B\right).\end{aligned} \tag{21}$$

Then the desired angular acceleration can be obtained as

$$\boldsymbol{\alpha}_r^B = \begin{bmatrix}-\boldsymbol{h}_\alpha\cdot\boldsymbol{y}_B, & \boldsymbol{h}_\alpha\cdot\boldsymbol{x}_B, & \ddot{\psi}_r\boldsymbol{z}_W\cdot\boldsymbol{z}_B\end{bmatrix}^T. \tag{22}$$

Note that in (20) to (22) we use the current attitude $\{\boldsymbol{x}_B,\ \boldsymbol{y}_B,\ \boldsymbol{z}_B\}$, angular velocity $\boldsymbol{\Omega}$, and collective thrust T instead of their references. As such, the drone follows the reference jerk $\dddot{\boldsymbol{\xi}}_r$ and snap $\ddddot{\boldsymbol{\xi}}_r$ even though its attitude, angular rates, and collective thrust are different from the reference.

Above derivations for $\boldsymbol{\Omega}_r^B$ and $\boldsymbol{\alpha}_r^B$ need the value of collective thrust T and its derivatives. While T can be calculated from (4)(5) with measured rotor speed $\boldsymbol{\omega}$, $\dot{T}$ and $\ddot{T}$ are unable to be directly measured. For this reason, we approximate them by using reference jerk $\dddot{\boldsymbol{\xi}}_r$ and snap $\ddddot{\boldsymbol{\xi}}_r$. Multiplying (dot-product) both sides of (18) by $\boldsymbol{z}_B$, we have

$$\dot{T} = m\dddot{\boldsymbol{\xi}}_r\cdot\boldsymbol{z}_B \tag{23}$$

and its derivative

$$\ddot{T} = m\ddddot{\boldsymbol{\xi}}_r\cdot\boldsymbol{z}_B + m(\boldsymbol{\Omega}\times\boldsymbol{z}_B)\cdot\dddot{\boldsymbol{\xi}}_r. \tag{24}$$

Equation (23) and (24) are then substituted into (18)-(22) to calculate $\boldsymbol{\Omega}_r^B$ and $\boldsymbol{\alpha}_r^B$.

3) Tilt-Prioritized Attitude Control: Quadrotors use rotor drag torques to achieve heading control. The control effectiveness of heading is around one order of magnitude lower than pitch and roll. As a consequence, heading control is prone to cause motor saturations. Fortunately, the thrust orientation of a quadrotor (namely tilt) is independent of its heading angle. Thus tilt-prioritized control has been proposed in [30] that regulates the reduced-attitude (pitch and roll) error $\boldsymbol{q}_e$ and yaw error $\boldsymbol{q}_{yaw}$ separately as follows

$$\boldsymbol{q}_e = \boldsymbol{q}_d\otimes\boldsymbol{q}^{-1} = [q_{e,w},\ q_{e,x},\ q_{e,y},\ q_{e,z}]^T, \tag{25}$$

$$\tilde{\boldsymbol{q}}_{e,red} = \frac{1}{\sqrt{q_{e,w}^2 + q_{e,z}^2}}\begin{bmatrix} q_{e,w}q_{e,x} - q_{e,y}q_{e,z} \\ q_{e,w}q_{e,y} + q_{e,x}q_{e,z} \\ 0\end{bmatrix}, \tag{26}$$

$$\tilde{\boldsymbol{q}}_{e,yaw} = \frac{1}{\sqrt{q_{e,w}^2 + q_{e,z}^2}}\begin{bmatrix}0 & 0 & q_{e,z}\end{bmatrix}^T. \tag{27}$$

Subsequently, the desired angular acceleration can be obtained by the following attitude control law:

$$\begin{aligned}\boldsymbol{\alpha}_d^B = k_{q,red}\tilde{\boldsymbol{q}}_{e,red} + k_{q,yaw}\mathrm{sgn}(q_{e,w})\tilde{\boldsymbol{q}}_{e,yaw} \\ + K_\Omega(\boldsymbol{\Omega}_r^B - \boldsymbol{\Omega}^B) + \boldsymbol{\alpha}_r^B\end{aligned} \tag{28}$$

where $k_{q,red}$ and $k_{e,yaw}$ are positive gains for reduced-attitude and yaw control respectively. Setting a relatively high $k_{q,red}$

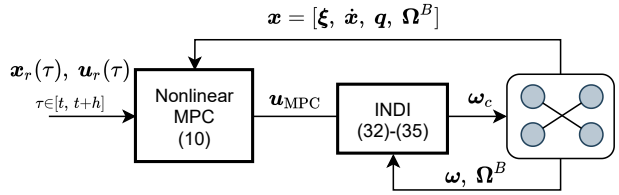


Fig. 3: The control diagram of the model predictive controller with an INDI inner-loop controller.

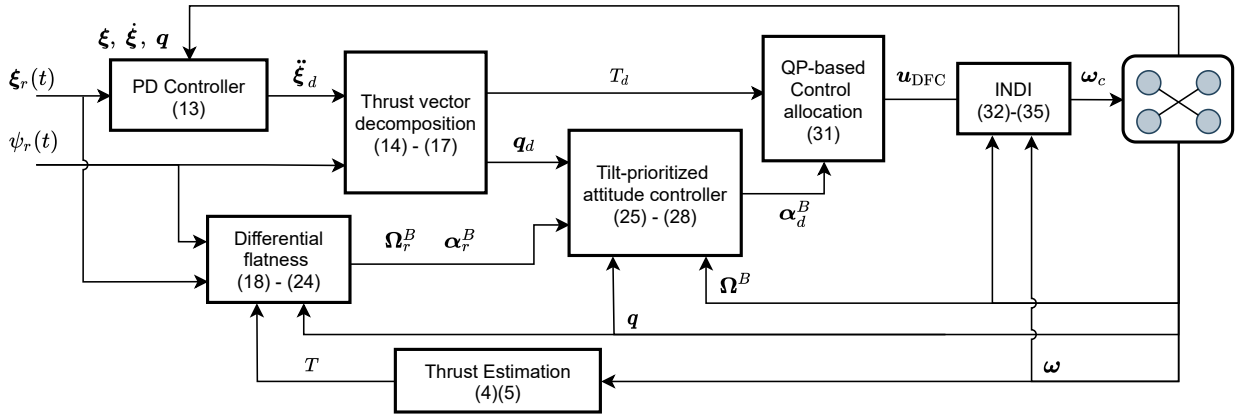


Fig. 4: The control diagram of the differential-flatness-based controller, combined with an INDI inner-loop controller.

over $k_{e,yaw}$ is helpful in improving position tracking accuracy while preventing input saturations. $\boldsymbol{\alpha}_r^B$ in (28) performs as a feed-forward term. In real-world experiments, we find that the performance improvement of adding this term is negligible.

4) Quadratic-Programming-Based Control Allocation: The control allocation module generates thrust commands of each individual rotor from desired collective thrust T_d and angular acceleration $\boldsymbol{\alpha}_d^B$. The same as NMPC, we also neglect the $\boldsymbol{G}_2$ and $\boldsymbol{G}_3$ terms in (4). Then from (3) and (4), we obtain the direct-inversion control allocation:

$$\boldsymbol{u} = \boldsymbol{G}_1^{-1} \begin{bmatrix} T_d \\ \boldsymbol{I}_v \boldsymbol{\alpha}_d^B + \boldsymbol{I}_v \times \boldsymbol{I}_v \boldsymbol{\Omega}^B \end{bmatrix}, \tag{29}$$

$$\boldsymbol{u}_{\mathrm{DFBC}} = \max\left(\boldsymbol{u}_{\min},\ \min(\boldsymbol{u},\ \boldsymbol{u}_{\max})\right) \tag{30}$$

This, however, does not consider input limits and may cause loss of control. For instance, an excessive collective thrust command can saturate all motors and consequently invalidate the attitude control.

An effective alternative to address saturations is the quadratic-programming based allocation that solves the following optimization problem:

$$\begin{aligned} \boldsymbol{u}_{\mathrm{DFBC}} = \underset{\boldsymbol{u}}{\operatorname{argmin}} \quad & \left\| \begin{bmatrix} T_d \\ \boldsymbol{I}_v \boldsymbol{\alpha}_d^B + \boldsymbol{I}_v \times \boldsymbol{I}_v \boldsymbol{\Omega}^B \end{bmatrix} - \boldsymbol{G}_1 \boldsymbol{u} \right\|_{\boldsymbol{W}} \\ s.t. \quad & \boldsymbol{u} \in [\boldsymbol{u}_{\min}\ \boldsymbol{u}_{\max}], \end{aligned} \tag{31}$$

where $\boldsymbol{W} \in \mathbb{R}^{4\times 4}$ is a positive-definite diagonal weight matrix. Each diagonal entry respectively indicates the weight on the thrust, pitch, roll and yaw channels. Generally, setting a relatively high weight on pitch and roll (see Table. I) is advantageous to prevent quadrotor loss-of-control when motor saturations are inevitable (e.g., tracking dynamically infeasible trajectories). If the solution is originally within control bounds, the result is the same as the direct-inversion allocation from (30). As for the implementation details, we solve this quadratic programming problem using an Active-Set Method from the qpOASES solver [41].

C. Incremental Nonlinear Dynamic Inversion

After obtaining thrust commands from (10) or (31), one can use (5) to directly obtain the rotor speed command. However, the above-mentioned controllers neglects the unmodeled term $\boldsymbol{d}_\tau$ in the rotational dynamics (3), which are found detrimental to the overall control performance.

Modeling $\boldsymbol{d}_\tau$ is very challenging for real-world systems. Therefore, we resort to incremental nonlinear dynamic inversion (INDI), a sensor-based controller that uses instantaneous sensor measurement, instead of a model, to represent system dynamics, thus being robust against model uncertainties and external disturbances. The performance and robustness of INDI have been demonstrated in previous research (see, e.g., [14, 12, 15]) with proven stability [42].

We use INDI as the inner-loop controller of both NMPC and DFBC for fair comparisons. The hybridization of INDI and DFBC is similar to a related work [12], except for the attitude controller and control allocation introduced in the previous section. Here, a method is proposed to effectively combine INDI with NMPC to improve its robustness against rotational model uncertainties.

After knowing the single rotor thrust command $\boldsymbol{u}_{\mathrm{DFBC}}$ or $\boldsymbol{u}_{\mathrm{NMPC}}$ from (31) and (10) respectively, we can retrieve the desired collective thrust, and desired angular acceleration using (3) and (4), yielding

$$\begin{bmatrix} \hat{T}_d \\ \boldsymbol{I}_v \hat{\boldsymbol{\alpha}}_d^B + \boldsymbol{\Omega}^B \times \boldsymbol{I}_v \boldsymbol{\Omega}^B \end{bmatrix} = \boldsymbol{G}_1 \boldsymbol{u}_{\mathrm{DFBC/NMPC}}. \tag{32}$$

Note that, for the DFBC method, $\hat{T}_d$ and $\hat{\boldsymbol{\alpha}}^B$ are different from those derived from (14) and (28) if the optimal cost of (31) is non-zero. Then from INDI, we can get the desired body torque (see (30) and (31) of [12] for detailed derivations)

$$\boldsymbol{\tau}_d = \boldsymbol{\tau}_f + \boldsymbol{I}_v \left(\hat{\boldsymbol{\alpha}}_d^B - \dot{\boldsymbol{\Omega}}_f^B \right) \tag{33}$$

Hence, the effect of unmodeled $\boldsymbol{d}_\tau$ is captured by filtered angular acceleration measurement $\dot{\boldsymbol{\Omega}}_f^B$ and filtered body torque $\boldsymbol{\tau}_f$, where

$$\boldsymbol{\tau}_f = \bar{\boldsymbol{G}}_1 \boldsymbol{\omega}_f^{\circ 2} + \Delta t^{-1} \bar{\boldsymbol{G}}_2 \left(\boldsymbol{\omega}_f - \boldsymbol{\omega}_{f,k-1} \right) \tag{34}$$

is calculated from rotor speed measurements. $\boldsymbol{\Omega}_f^B$ and $\boldsymbol{\omega}_f$ are low-pass filtered body-rate and rotor speeds with the same cut-off frequencies. Thus they have a similar amount of delay and can be synchronized. In this paper, we use a second-order Butterworth filter with a 12 Hz cut-off frequency. Note that we use subscript $k-1$ to indicate the last sampled variable, and Δt is the sampling interval. $\bar{\boldsymbol{G}}_1$ and $\bar{\boldsymbol{G}}_2$ respectively indicate matrices formed by the last three rows of $\boldsymbol{G}_1$ and $\boldsymbol{G}_2$.

Then from (14), the following equation is obtained to solve rotor speed command $\boldsymbol{\omega}_c$:

$$\begin{bmatrix} \hat{T}_d \\ \boldsymbol{\tau}_d \end{bmatrix} = \boldsymbol{G}_1 \boldsymbol{\omega}_c^{\circ 2} + \Delta t^{-1} \boldsymbol{G}_2 \left(\boldsymbol{\omega}_c - \boldsymbol{\omega}_{c,k-1} \right), \tag{35}$$

with the only unknown $\boldsymbol{\omega}_c$ which can be solved numerically. Differently from [12], the motor time constant is not needed in (35).

The INDI inner-loop controller converts the original thrust inputs from the high-level controller to rotor speed commands. Because this conversion is through algebraic equations, system delay typically seen in the cascaded control structures can be effectively circumvented. The advantage of INDI over classical PID inner-loop controller will be demonstrated in Sec. VI.

V. Simulation Experiments

In a set of controlled experiments in simulation, we examine DFBC and NMPC to address the following research questions: (i) Given the computational budget and a well-identified model, which control approach achieves better tracking performance? (ii) How do these two approaches perform in tracking dynamically *infeasible* trajectories, which inevitably lead to actuator saturation? (iii) How do these two approaches perform in simulated practical situations with a model mismatch, external disturbances, state estimation latency? Note that in this section, both methods are tested with the augmentation of the INDI inner-loop controller. In this section, we use NMPC and DFBC to represent those with INDI and an aerodynamic drag model for readability.

A. Simulation Setup

The simulator uses a 4th-order Runge-Kutta integrator to propagate quadrotor dynamic equations (1)-(4) at 500 Hz. The DFBC and INDI run at 300 Hz while NMPC runs at 100 Hz. These frequencies are selected for consistency with real-world experimental setups which will be introduced in Sec. VI. Rotor speed commands generated by the controllers pass through a low-pass filter with a 30 ms time constant to simulate the motor dynamics. By assuming the controller knows a perfect aerodynamics model, we include the aerodynamic drag model (9) in the simulator. The parameters of the simulated drone are given in Table II. Both flight controllers and the simulator are programmed in C++, with ACADO and qpOASES as external libraries. A wrapper in ROS environment is also written which allows data logging, interfacing, and network communication.

TABLE I: Controller gains and parameters

NMPC		DFBC	
$\boldsymbol{Q}_\xi$	diag(200, 200, 500)	$\boldsymbol{K}_\xi$	diag(10, 10, 10)
$\boldsymbol{Q}_v$	diag(1, 1, 1)	$\boldsymbol{K}_v$	diag(6, 6, 6)
$\boldsymbol{Q}_q$	diag(5, 5, 200)	k_{qx}, k_{qy}, k_{qz}	150, 150, 3
$\boldsymbol{Q}_\Omega$	diag(1, 1, 1)	$\boldsymbol{K}_\Omega$	diag(20, 20, 8)
$\boldsymbol{Q}_u$	diag(6, 6, 6, 6)	$\boldsymbol{W}$	diag(0.001, 10, 10, 0.1)
dt	50 ms		
N	20		

TABLE II: Quadrotor Configurations

Parameter(s)		Value(s)
m	[kg]	0.75
l	[m]	0.14
β	[deg]	56
$\boldsymbol{I}_v$	[gm^2]	diag(2.5, 2.1, 4.3)
$(u_{\min}, u_{\max})$	[N]	(0, 8.5)
c_q	[Nm^2]	$2.37e^{-8}$
c_t	[N^2]	$1.51e^{-6}$
$(k_{d,x},\ k_{d,y},\ k_{d,z})$	[kg/s]	(0.26, 0.28, 0.42)
k_h	[kg/m]	0.01

B. Reference Trajectories

Multiple elliptical reference trajectories on both horizontal and vertical planes are generated for testing the tracking performance. The results of tracking these simple trajectory primitives can further reflect the performance of tracking more complex 3D trajectories. These trajectories are parameterized by the maximum velocity $V_{\max}$, maximum acceleration $a_{\max}$, and ellipticity n. Specifically, the horizontal reference trajectories are expressed as:

$$\boldsymbol{\xi}_r(t) = \left[r_{\max} \sin(kt),\ r_{\min} \cos(kt),\ 5 \right]^T; \tag{36}$$

and the vertical reference trajectories are

$$\boldsymbol{\xi}_r(t) = \left[r_{\max} \sin(kt),\ 0,\ 5 + r_{\min} \cos(kt) \right]^T \tag{37}$$

where

$$r_{\max} = V_{\max}^2 / a_{\max},\ k = a_{\max}/V_{\max},\ r_{\min} = r_{\max}/n. \tag{38}$$

The heading angle of each horizontal reference trajectory is selected to always point at the center of the ellipse; for vertical trajectories, the reference heading angles are kept constant. Specifically, 144 reference trajectories are generated by combining parameters $a_{\max} \in \{10,\ 20,\ 30,\ 40,\ 50,\ 60\}$ m/s^2, $V_{\max} \in \{5,\ 10,\ 15,\ 20\}$ m/s, $n \in \{1,\ 2,\ 5\}$.

Tracking some of these trajectories may require the quadrotor to exceed its thrust limitations. These trajectories are referred to as dynamically *infeasible*. In many time-critical applications, such as autonomous drone racing, rotors need to reach their full thrust limits to fully exploit the agility of the

platform and achieve faster lap times. Designing such an agile time-optimal trajectory requires a perfect model that captures real thrust limits and aerodynamic effects in high-speed flights. Without this model, the generated trajectory may exceed the quadrotor capabilities and becomes dynamically *infeasible* to accurately follow. Since such an accurate model is usually unattainable, studying the performance when tracking these *infeasible* trajectories is necessary.

For the simulated quadrotor with the configurations given in Table II, there are 68 *infeasible* trajectories and 76 *feasible* trajectories. They are determined by being tracked by a modified NMPC without thrust limits imposed. If the applied thrust exceeds the maximum thrust of the real drone, the trajectory is marked as *infeasible*. The feasibility of each reference trajectory is given in Table III

TABLE III: Dynamical feasibility of reference trajectories. (*feasible*:✓; *infeasible*:✗)

(a) Horizontal trajectories.

$a_{\max}$ [m/s^2]	$n=1$				$n=2$				$n=5$			
	$V_{\max}$ [m/s]				$V_{\max}$ [m/s]				$V_{\max}$ [m/s]			
	5	10	15	20	5	10	15	20	5	10	15	20
10	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
20	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓
30	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✓	✓
40	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗	✓
50	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
60	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

(b) Vertical trajectories.

$a_{\max}$ [m/s^2]	$n=1$				$n=2$				$n=5$			
	$V_{\max}$ [m/s]				$V_{\max}$ [m/s]				$V_{\max}$ [m/s]			
	5	10	15	20	5	10	15	20	5	10	15	20
10	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
20	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
30	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
40	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
50	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
60	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

C. Evaluation Criteria

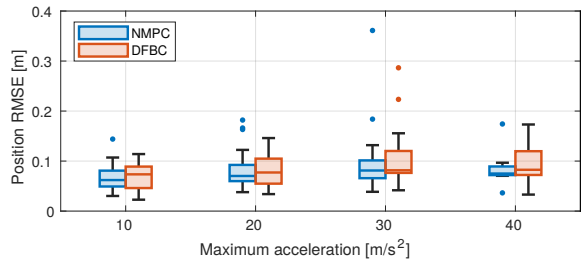
In the following comparisons, we use the root mean square error (RMSE) of position and heading as the precision metric of a method. The crash rate is another criterion to show the robustness of a method. A certain flight is defined as *Crashed* if its position $\boldsymbol{\xi}$ violates the following spatial constraint at an arbitrary instant:

$$\inf\{\boldsymbol{\xi}_r(t)\} - \boldsymbol{b} \leq \boldsymbol{\xi} \leq \sup\{\boldsymbol{\xi}_r(t)\} + \boldsymbol{b}. \tag{39}$$

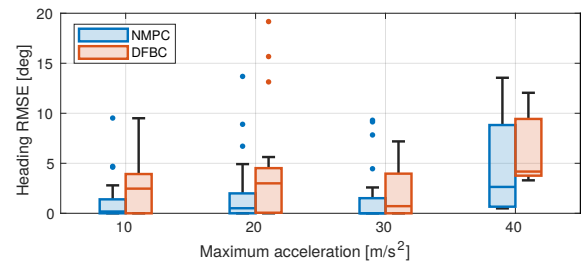
We select $\boldsymbol{b} = [5,\ 5,\ 5]^T$ meters in this study.

D. Tracking Dynamically Feasible Trajectories

First of all, we compare the performance of NMPC and DFBC in tracking 76 *feasible* trajectories. We perform the test in an ideal condition with perfect model knowledge and state estimates. Since only *feasible* trajectories are tracked, there is no saturation of single rotor thrusts and both methods succeed in tracking all trajectories without a single crash.

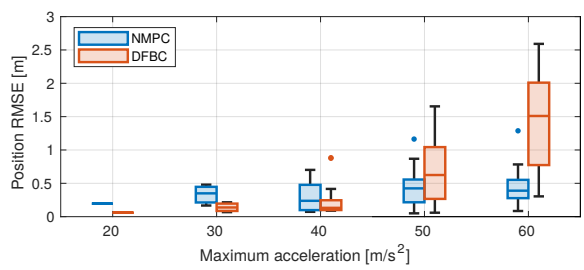


(a)

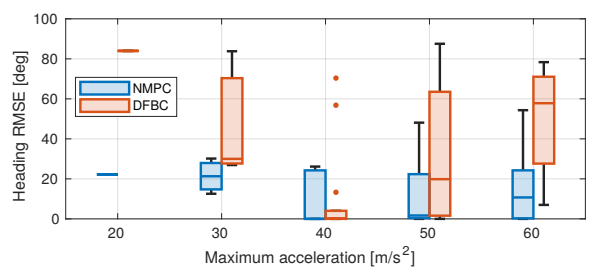


(b)

Fig. 5: Boxplot of tracking error (RMSE) in tracking different dynamically *feasible* trajectories (76 in total) categorized by maximum accelerations.



(a)



(b)

Fig. 6: Boxplot of tracking error (RMSE) in tracking different dynamically *infeasible* trajectories (68 in total) categorized by maximum accelerations. Crashed flights are removed from these analyses (see Fig. 7 for the crash rate).

In this set of tests, we fine-tune the parameters of both methods (listed in Table. I) such that they achieve a similar position tracking error. Fig. 5 compares the boxplots of NMPC and DFBC when tracking trajectories with different reference maximum accelerations. Both methods have similar position RMSE in these flights. As for the heading tracking, NMPC has an average heading RMSE of 2.0 deg, which is better than 5.8 deg of DFBC.

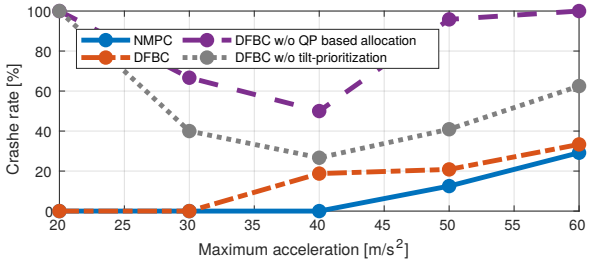


Fig. 7: Crash rates of NMPC, DFBC, and DFBC with different setups: DFBC without quadratic-programming based allocation, DFBC without tilt-prioritized attitude controller but using the geometric attitude controller.

E. Tracking Dynamically Infeasible Trajectories

1) Tracking Accuracy: Fig. 6 shows the box plot of position and heading RMSE of NMPC and DFBC in tracking the previously generated 68 *infeasible* trajectories. Crashed flights are excluded from this plot (see Fig. 7 for the crash rate). We find that the DFBC method shows smaller position RMSE in tracking trajectories with lower acceleration. However, as the reference acceleration grows, NMPC significantly outperforms DFBC. In general, NMPC outperforms DFBC by 48% on position tracking (0.40 m vs. 0.77 m). As for the heading tracking, NMPC has significantly smaller (62%) RMSE than the DFBC method (12.7 deg vs. 33.4 deg). The advantage of NMPC over DFBC becomes particularly relevant if the reference acceleration exceeds the maximum thrust ($\sim$ 5g) of the tested quadrotor.

2) Crash Rates: Fig. 7 compares the crash rates of DFBC and NMPC in tracking all the *infeasible* trajectories. NMPC (solid-blue) outperforms DFBC (red-dash-dot), especially in tracking trajectories with high reference accelerations.

Here, we also demonstrate contributions of the tilt-prioritized attitude controller and the quadratic-programming based allocation presented in Sec. IV. We replace them respectively with the conventional *geometric attitude controller* [23] and the *direct-inversion allocation* (30). The resultant crash rates are shown in Fig. 7 as well. It is obvious that the DFBC method tested in this study shows a significantly lower crash rate in tracking *infeasible* trajectories.

F. Robustness Study

While previous simulations are carried out in an ideal condition, this section studies the robustness of NMPC and DFBC methods in the following non-ideal conditions:

- Model mismatch, including the center of gravity bias, mass mismatch, thrust and torque coefficients model mismatch, and error of aerodynamic drag model.
- External disturbances. In details, we simulate constant external forces along $\boldsymbol{x}_I$, or external torques along both $\boldsymbol{x}_B$ and $\boldsymbol{y}_B$. These disturbances last for 5 seconds.
- Position, velocity, and attitude estimation latency. This aims at studying the effect of latency from such as signal transmission, state estimation algorithms, and sensors. As angular rates are measured directly by IMU with negligible latency, we only study the latency on pose and velocity estimates.

These robustness studies are conducted using only the *feasible* trajectories since NMPC already shows its advantage over DFBC in tracking *infeasible* trajectories. We set the position RMSE as 5.0 m and heading RMSE as 90 deg for those crashed flights.

Table IV and Table V respectively show the results without and with INDI inner-loop. We have the following observations.

NMPC shows substantially higher robustness than DFBC against model uncertainties and disturbances on rotational dynamics due to CoG bias or external moments. Adding an INDI inner-loop controller can improve the robustness against these uncertainties. Even so, NMPC still slightly outperforms DFBC when both are augmented by an INDI inner-loop controller.

Unlike the rotational dynamics, NMPC shows a higher crash rate while experiencing model uncertainties and disturbances acting on the translational dynamics. For example, when the real mass is 30% higher, both controllers generate fewer thrusts than required to track the trajectory, resulting in a constant position tracking error. With this tracking error, NMPC fails to converge in over 10% of flights and crashes the drone. The same reason also explains the significantly higher crash rate of NMPC in the presence of large external disturbances. Fig. 8 shows an example where NMPC failed in converging and crashed the drone after experiencing a 10 N external force disturbance. Note that adding INDI inner-loop does not improve the performance of either controller to reject disturbances in the translational dynamics. A previous research [12] implemented INDI in the position control (INDI outer-loop) for DFBC by virtue of its cascaded structure and improved its translational robustness. On the contrary, hybridizing INDI outer-loop with NMPC seems challenging owing to its non-cascaded nature.

The two methods also perform differently in the presence of system latency. While NMPC slightly outperforms DFBC when the system latency is lower than 30 ms, as the latency grows, NMPC shows a much higher crash rate (68.0%) than DFBC (6.7%). As Fig. 9 shows, we repeat the tests under 30 ms and 50 ms latency with a high-gain setup and a low-gain setup. In both setups, these gains are tuned such that both controllers have identical average position RMSE in the ideal condition. As is expected, reducing gains will alleviate the effect of estimation latency. Interestingly, we observe that NMPC is more sensitive to the gain selections when system latency is larger than 30 ms.

G. Effect of Controller Parameters

Controller performance is dominated by the parameters or gains. However, finding optimal controller gains can be tedious and highly dependent on the drone parameters. Thus this section performs a sensitivity analysis to provide insights into the effect of controller parameters on the tracking performance. We set the control parameters given in Table I as the baseline, and compare the tracking performance with altered parameters with respect to the baseline.

TABLE IV: Tracking performance comparisons in different types of non-ideal conditions commonly seen in practice **without INDI inner loop.** Each number shows the mean and standard deviation (SD) across tracking 76 dynamically *feasible* trajectories.

	Position RMSE [m] (mean ± SD)		Heading RMSE [deg] (mean ± SD)		Crash Rate [%]	
	NMPC	DFBC	NMPC	DFBC	NMPC	DFBC
Baseline	0.084 ± 0.049	0.084 ± 0.041	**2.13** ± 3.38	5.91 ± 11.90	0	0
+50% Drag	**0.183** ± 0.066	0.194 ± 0.084	**2.11** ± 3.35	8.56 ± 15.46	0	0
+100% Drag	**0.286** ± 0.109	0.305 ± 0.139	**2.25** ± 3.38	9.07 ± 11.44	0	0
-30% Mass	**0.320** ± 0.105	0.477 ± 0.109	**2.27** ± 3.38	11.26 ± 17.77	0	0
+30% Mass	0.809 ± 1.455	**0.687** ± 1.001	11.77 ± 27.21	**7.21** ± 14.11	10.7	**0**
10% CoG bias	**0.264** ± 0.094	1.499 ± 1.802	**2.67** ± 3.42	25.95 ± 35.69	**0**	20
15% CoG bias	**0.464** ± 0.557	3.039 ± 2.216	**4.58** ± 10.94	53.62 ± 41.95	**1.3**	56
-30% c_t	**0.939** ± 1.415	0.998 ± 1.216	12.30 ± 27.13	**8.79** ± 18.01	10.7	**0**
+30% c_t	**0.245** ± 0.085	0.36 ± 0.076	**2.05** ± 3.34	6.63 ± 10.87	0	0
-30% c_q	0.085 ± 0.052	**0.084** ± 0.041	**2.03** ± 3.37	6.23 ± 13.03	0	0
+30% c_q	0.084 ± 0.051	0.084 ± 0.04	**2.07** ± 3.38	5.81 ± 11.81	0	0
5N Ext. force	**0.252** ± 0.042	0.302 ± 0.031	**5.08** ± 3.58	17.42 ± 14.59	0	0
10N Ext. force	0.716 ± 1.019	**0.689** ± 0.524	**12.59** ± 19.07	24.59 ± 16.28	5.3	**1.3**
15N Ext. force	1.87 ± 1.892	**1.391** ± 1.069	**32.13** ± 35.38	32.54 ± 20.58	26.7	**6.7**
0.1Nm Ext. moment	0.153 ± 0.038	0.27 ± **0.098**	**3.26** ± 3.35	12.04 ± 13.45	0	0
0.2Nm Ext. moment	**0.319** ± 0.545	2.538 ± 2.079	**5.85** ± 10.71	54.89 ± 31.08	**1.3**	41.3
0.3Nm Ext. moment	**0.566** ± 0.907	4.949 ± 0.439	**10.87** ± 17.27	89.16 ± 6.88	**4.0**	98.7
10ms Latency	**0.085** ± 0.065	0.097 ± 0.099	**2.07** ± 3.34	5.79 ± 11.05	0	0
30ms Latency	**0.158** ± 0.183	**0.333** ± 0.976	5.04 ± 10.47	13.73 ± 25.87	**0**	4.0
50ms Latency	3.45 ± 2.217	**0.669** ± 1.247	65.00 ± 38.65	**54.82** ± 30.02	66.7	**5.3**

TABLE V: Tracking performance comparisons in different types of non-ideal conditions commonly seen in practice **with INDI inner loop.** Each number shows the mean and standard deviation (SD) across tracking 76 dynamically *feasible* trajectories.

	Position RMSE [m] (mean ± SD)		Heading RMSE [deg] (mean ± SD)		Crash Rate [%]	
	NMPC	DFBC	NMPC	DFBC	NMPC	DFBC
Baseline	**0.082** ± 0.047	0.085 ± 0.043	**2.03** ± 3.35	5.83 ± 11.40	0	0
+50% Drag	**0.184** ± 0.067	0.194 ± 0.084	**2.10** ± 3.33	8.48 ± 14.50	0	0
+100% Drag	**0.288** ± 0.11	0.306 ± 0.139	**2.25** ± 3.32	9.32 ± 11.99	0	0
-30% Mass	**0.320** ± 0.106	0.477 ± 0.108	**2.26** ± 3.36	10.42 ± 14.19	0	0
+30% Mass	0.747 ± 1.372	**0.686** ± 0.999	10.60 ± 25.66	**7.11** ± 13.97	9.3	**0**
10% CoG bias	**0.081** ± 0.047	0.099 ± 0.104	**2.09** ± 3.32	6.46 ± 13.65	0	0
15% CoG bias	**0.083** ± 0.051	0.378 ± 1.117	**2.05** ± 3.30	10.70 ± 22.52	**0**	5.3
-30% c_t	1.362 ± 1.764	**1.004** ± 1.212	19.98 ± 33.88	**10.06** ± 19.06	18.7	**1.3**
+30% c_t	**0.244** ± 0.085	0.359 ± 0.076	**2.13** ± 3.52	6.14 ± 9.58	0	0
-30% c_q	**0.082** ± 0.049	0.086 ± 0.047	**2.08** ± 3.35	6.01 ± 11.69	0	0
+30% c_q	**0.082** ± 0.046	0.085 ± 0.046	**2.07** ± 3.34	5.83 ± 11.61	0	0
5N Ext. force	**0.254** ± 0.051	0.302 ± 0.033	**5.03** ± 3.48	17.58 ± 14.77	0	0
10N Ext. force	0.725 ± 1.018	**0.690** ± 0.524	**12.38** ± 19.02	24.81 ± 16.28	5.3	**1.3**
15N Ext. force	1.872 ± 1.891	**1.393** ± 1.070	**32.14** ± 35.35	32.97 ± 20.80	26.7	**6.7**
0.1Nm Ext. moment	**0.083** ± 0.048	0.087 ± 0.049	**2.49** ± 3.22	7.95 ± 12.46	0	0
0.2Nm Ext. moment	**0.087** ± 0.051	0.113 ± 0.088	**3.19** ± 3.35	13.65 ± 17.25	0	0
0.3Nm Ext. moment	**0.298** ± 0.962	0.764 ± 1.663	**8.40** ± 17.29	26.93 ± 29.41	**4.0**	13.3
10ms Latency	**0.087** ± 0.088	0.099 ± 0.106	**2.21** ± 3.44	5.73 ± 11.65	0	0
30ms Latency	0.280 ± 0.802	**0.265** ± 0.800	**8.39** ± 20.12	14.34 ± 25.85	2.7	2.7
50ms Latency	3.480 ± 2.235	**0.694** ± 1.294	64.02 ± 39.96	**52.29** ± 29.12	68.0	**6.7**

1) Nonlinear MPC: Fig. 10 shows the effect of changing elements of the weight matrix $\boldsymbol{Q}$. The position tracking performance is highly correlated with $\boldsymbol{Q}_\xi$ and $\boldsymbol{Q}_v$, rather than attitude related weights $\boldsymbol{Q}_q$ and $\boldsymbol{Q}_\Omega$. In fact, increasing weight on body rates even degrades the tracking performance. Among all the parameters, the position weight $\boldsymbol{Q}_\xi$ plays the most important role. A larger penalty on the position error results in a smaller tracking error. However, as $\boldsymbol{Q}_\xi$ continues growing, the NMPC starts to destabilize the drone. We believe that this is due to the presence of actuator dynamics in the simulation, which introduces system delay and causes instability with high gain controllers. Note that in real-world experiments, system latency can be larger due to state estimation algorithms and signal transmissions, which further reduces the permitted maximum $\boldsymbol{Q}_\xi$.

2) Differential Flatness Based Control: The controller gains of DFBC consists of position, attitude gains and weight matrix for control allocation. The gains are selected to make the position error $\boldsymbol{\xi}_e = \boldsymbol{\xi} - \boldsymbol{\xi}_r$ a second-order system:

$$\ddot{\boldsymbol{\xi}}_e + 2\zeta_\xi \omega_{n,\xi} \dot{\boldsymbol{\xi}}_e + \omega_{n,\xi}^2 \boldsymbol{\xi}_e = 0 \tag{40}$$

where ζ_ξ and $\omega_{n,\xi}$ are damping ratio and natural frequency, respectively. Then from (13), we can calculate the position

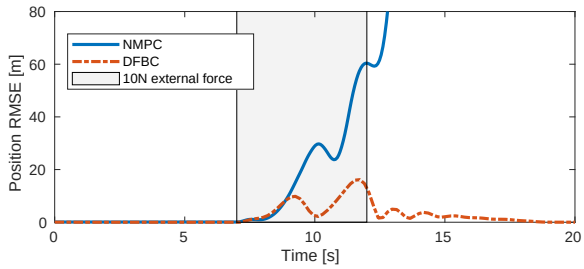


Fig. 8: Position tracking error for a loop trajectory ($V_{max} = 15m/s$, $a_{max} = 40\text{m/s}^2$, $n = 1$). Grey area indicates the period with 10 N lateral disturbances acting on the drone. NMPC failed to converge and crashed the drone, while the DFBC method succeeds in recovering the drone after the external disturbance disappeared.

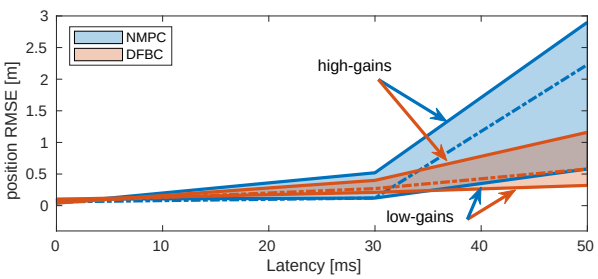


Fig. 9: Position RMSE of NMPC and DFBC in the presence of different amounts of estimation latency, with different sets of control gains. Results using gains of previous studies are shown in dash-dot lines. NMPC shows higher sensitivity to the gain selections when estimation latency appears.

control gains as follows:

$$\boldsymbol{K}_\xi = \text{diag}(\omega_{\text{n},\xi}^2,\ \omega_{\text{n},\xi}^2,\ \omega_{\text{n},\xi}^2),\ \boldsymbol{K}_v = 2\zeta_\xi\sqrt{\boldsymbol{K}_\xi}. \tag{41}$$

Similarly, the attitude gains can be designed as:

$$k_{q,x} = k_{q,y} = \omega_{n,xy}^2, k_{q,z} = \omega_{n,z}^2, \tag{42}$$

$$\boldsymbol{K}_\Omega = 2\zeta_q \text{diag}(\sqrt{k_{q,x}}, \sqrt{k_{q,y}}, \sqrt{k_{q,z}}). \tag{43}$$

where ζ_q is the damping ratio, $\omega_{n,xy}$ and $\omega_{n,z}$ are desired natural frequencies of reduced attitude and yaw errors respectively.

Fig. 11 presents the relationship between tracking error and closed-loop natural frequency. Both time constants and

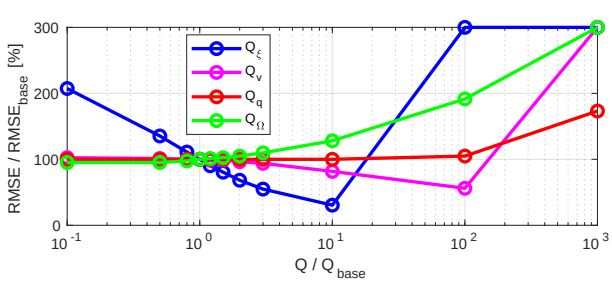


Fig. 10: Position tracking RMSE of the NMPC method in different control parameters. The vertical axis represents the ratio between RMSE and that from a baseline parameter setup. The horizontal axis represents the ratio between the control parameter and the baseline parameter. The RMSE of those crashed flights are clamped at 3 times of $\text{RMSE}_{\text{base}}$

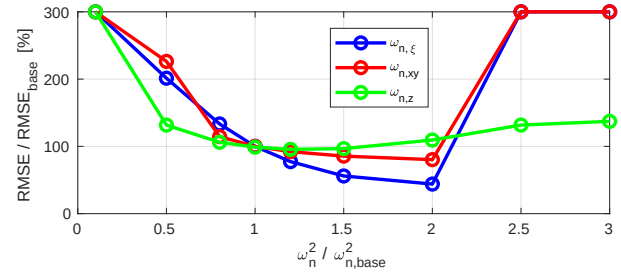


Fig. 11: Position tracking RMSE of the DFBC method in different control gain settings. The vertical axis represents the ratio between RMSE and the RMSE from a baseline gain setup. The horizontal axis represents the ratio between the controller gains and the baseline gains. RMSE of the crashed flights are clamped at 3 times of $\text{RMSE}_{\text{base}}$

tracking errors are normalized by the baseline gains given in Table I. A higher frequency indicates a higher control gain, leading to less tracking error. However, high gains are also prone to suffer from system delay and subsequently cause instability.

Weighting matrix $\boldsymbol{W}$ is selected based on tuning in the simulation without sophisticated calculation. It aims at leaving more priority to the tilt control over collective thrust and yaw. An example of inappropriate selection of $\boldsymbol{W}$ is an identical matrix, which makes the QP-based allocation identical to a direct inversion. The performance in terms of crash rate has been compared in Fig. 7 necessity of using a well-tuned $\boldsymbol{W}$ in tracking dynamically *infeasible* trajectories.

VI. Real-World Experiments

We conduct the experiments on a quadrotor in an instrumented tracking arena. A set of aggressive trajectories are executed to compare the closed-loop tracking performance of NMPC and DFBC in the presence of joint effects such as model mismatch, state estimation error, and system latency. The real-world experiments also highlight the contributions of the INDI low-level controller, the aerodynamic force model, to both NMPC and DFBC methods.

A. Experimental Setup

The real-flight experiment is performed in an indoor arena equipped with a motion capture system (VICON), with a 30 $\times$ 30 $\times$ 8 m^3 tracking volume, as is shown in Fig. 1. The inertial and geometric parameters of the quadrotor are listed in Table II, which are the same for the simulation study in Sec. V. The flight software runs at 300 Hz on an onboard NVIDIA Jetson TX2 embedded computer. It includes the control algorithms (DFBC/NMPC and INDI inner-loop), and an extended Kalman filter fusing VICON (400 Hz) and IMU measurements (500 Hz) to obtain the full-state estimates. While DFBC runs at 300 Hz, the frequency of NMPC is limited to 100 Hz due to its computational complexity. The INDI inner-loop also runs at 300 Hz no matter which controller is being used. The onboard computer sends rotor speed commands of individual rotors to a low-level RadixFC board. The latter runs a customized firmware based on the open-source NuttX real-time operating

TABLE VI: Tracking performance of different methods in real-world experiments

References			Position Tracking RMSE [m]							Heading Tracking RMSE [deg]						
Type	$V_{\max}$ [m/s]	$a_{\max}$ [m/s^2]	NMPC+PID [4]	NMPC	NMPC+INDI	NMPC+INDI (w/o drag model)	DFBC	DFBC+INDI	DFBC+INDI (w/o drag model)	NMPC+PID [4]	NMPC	NMPC+INDI	NMPC+ INDI (w/o drag model)	DFBC	DFBC + INDI	DFBC+ INDI (w/o drag model)
Loop A	10.2	20.7	0.280	0.548	**0.068**	0.215	0.897	0.073	0.242	6.801	11.274	8.450	8.972	73.842	**4.411**	9.395
Loop B	13.4	35.9	0.369	0.930	**0.117**	0.261	-	0.125	0.252	11.618	22.193	**9.714**	10.478	-	10.496	11.489
Slant Loop A	8.2	12.6	0.241	0.458	**0.077**	0.156	0.535	0.109	0.163	6.951	19.795	**5.987**	6.870	17.755	7.521	8.032
Slant Loop B	12.9	30.8	0.447	0.777	**0.116**	0.217	-	0.138	0.305	7.729	18.382	**8.282**	8.816	-	9.171	10.047
Vertical Loop A	7.7	14.6	0.197	0.206	**0.069**	0.172	0.181	0.082	0.127	1.558	5.131	**1.108**	1.274	30.901	1.151	1.530
Vertical Loop B	11.5	32.9	0.482	0.350	**0.126**	0.314	-	0.177	0.196	19.000	6.019	2.415	**1.900**	-	15.495	8.632
Oscillate A	10.9	20.7	0.327	0.212	**0.058**	0.259	0.310	0.164	0.210	1.641	7.369	**2.029**	1.960	50.210	3.977	1.911
Oscillate B	14.0	36.5	0.375	0.290	**0.125**	0.377	-	0.129	0.294	2.708	9.228	**2.386**	2.902	-	2.763	3.821
Hairpin A	7.7	14.6	0.185	0.352	**0.024**	0.146	0.350	0.049	0.124	4.733	7.100	**3.745**	4.255	38.980	5.913	5.873
Hairpin B	11.5	32.9	0.390	0.479	**0.099**	0.274	1.797	0.117	0.166	7.965	15.748	**7.343**	8.017	68.139	10.163	8.004
Lemniscate A	9.5	13.6	0.225	0.324	**0.056**	0.184	0.423	0.068	0.216	3.576	9.074	3.689	4.109	37.235	**2.605**	2.243
Lemniscate B	14.2	36.0	0.314	0.564	0.130	0.290	-	**0.128**	0.287	7.335	15.952	6.280	6.659	-	**4.986**	5.403
Split-S	14.2	25.5	0.292	0.352	0.096	0.262	-	**0.087**	0.239	9.411	13.253	6.409	**5.259**	-	17.888	18.929
Racing Track A	11.9	22.1	0.280	0.424	0.108	0.212	0.552	**0.102**	0.240	6.806	10.434	6.566	7.706	37.731	**4.336**	5.390
Racing Track B	16.8	28.5	0.369	0.546	**0.130**	0.320	3.797	0.152	0.359	11.930	14.672	9.318	8.805	57.810	**8.249**	7.719
Racing Track C	19.4	37.3	0.712	0.758	**0.238**	0.329	-	0.259	0.480	11.235	19.936	11.754	**9.876**	-	14.695	15.498
Mean			0.343	0.473	**0.102**	0.249	0.982	0.122	0.244	7.562	12.848	**5.967**	6.116	45.845	7.739	7.745
Standard deviation			0.130	0.207	0.067	**0.048**	1.161	0.164	0.051	4.499	5.434	**3.054**	3.136	18.175	4.768	4.964

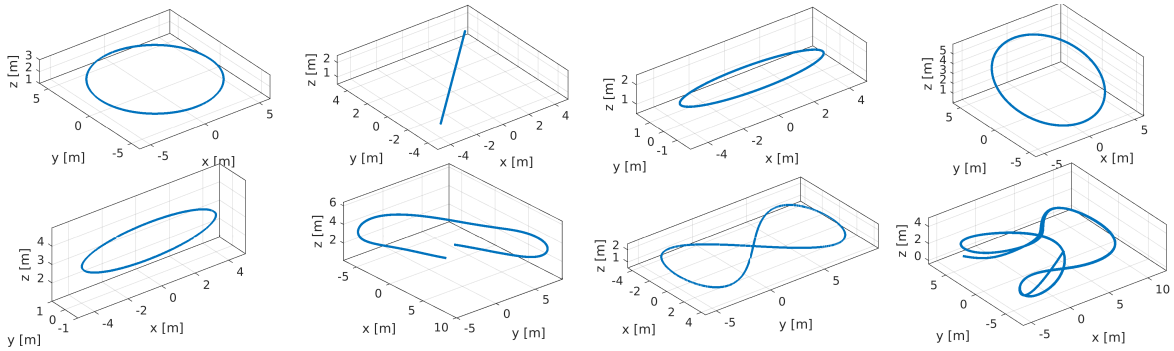


Fig. 12: Tested trajectories in real-world experiments. From top-left to bottom right, they are: Loop, Oscillate, Hairpin, Slant-loop, Vertical-loop, Split-S, Lemniscate, Racing-trajectory.

TABLE VII: Performance in tracking three dynamically infeasible trajectories in real-world experiments. Position and heading RMSE comparisons between NMPC and DFBC with INDI inner-loop.

Reference	$V_{\max}$ [m]	$a_{\max}$ [m/s^2]	Position RMSE [m]		Heading RMSE [deg]	
			NMPC +INDI	DFBC +INDI	NMPC +INDI	DFBC +INDI
Vertical Loop C	12.1	48.5	**0.318**	0.919	**2.469**	25.608
Loop C	15.7	49.3	**0.762**	1.829	**13.439**	25.876
Lemniscate C	19.0	54.5	**0.570**	1.237	**11.680**	53.262
Average RMSE			**0.550**	1.328	**9.196**	34.915

system at 500 Hz to perform closed-loop rotor speed control, and sends rotor speed and IMU measurements to the onboard computer.

Experiments are conducted in tracking different reference trajectories, ranging from simple loop trajectories to FPV drone racing tracks (see Table. VI). Trajectories with the same 3D shape but different velocities and accelerations are also tested. The 3D paths of these trajectories are illustrated in Fig. 12.

We evaluated NMPC and DFBC, with and without the INDI inner-loop controller. NMPC with a PID low-level controller is also tested for comparison, implemented to track time-optimal trajectory in [4]. In the following, these methods consider aerodynamic drag models by default, unless further mentioned. The position tracking RMSE and heading tracking RMSE are listed in Table. VI and Table. VII.

B. Contribution of the INDI and Drag Model

Table VIII compares the average RMSE of both methods and shows the effect of the INDI inner-loop controller, and the effect of the aerodynamic drag model. For NMPC and DFBC, neglecting aerodynamic drag would increase position tracking error by 144% percent and 122%, respectively. In contrast to the aerodynamic drag model, removing the INDI low-level controller influences more on the tracking accuracy. For NMPC, we see more than 364% and 115% increase in position and heading RMSE if INDI is not used. For DFBC, removing INDI leads to a more severe consequence: more than 705% and 492% of position and heading RMSE increase. In fact, DFBC without INDI cannot successfully track some of these trajectories without crashing the drone. These results indicate that, an adaptive/robust inner-loop controller plays a

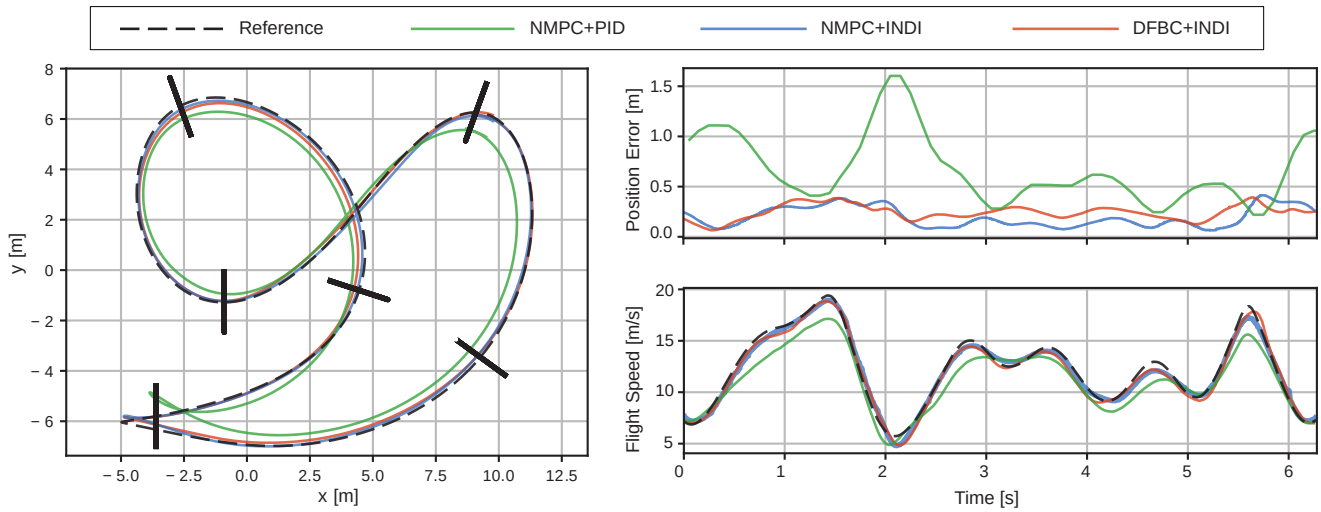


Fig. 13: Tracking performance of the *Race Track C* with a maximum speed up to 20 $\mathrm{m/s}$. NMPC and DFBC with INDI inner-loop show similar tracking performance. Both outperform the state-of-the-art NMPC with a PID inner-loop controller [4]

more important role than the drag model in accurately tracking aggressive trajectories.

We also make comparisons against the control method used in [4], which uses PID as the inner-loop controller of NMPC. This comparison is made in tracking a time-optimal trajectory generated off-line using the algorithm proposed in [4]. In this task, the quadrotor needs to fly through multiple gates in a predefined order. Fig. 13 demonstrates the tracking results, including the 3D path and the position error. Clearly, both NMPC and DFBC augmented by INDI significantly outperform this NMPC with PID inner-loop controller. Comparative results on other reference trajectories can be found in Table VI. On average, the proposed NMPC with the INDI approach shows a position RMSE of 0.102 m, which is 70% lower than the position RMSE of NMPC with PID low-level controller (0.343 m).

TABLE VIII: Position and heading tracking RMSE of NMPC and DFBC methods. Adding INDI as the inner-loop improves the performance significantly for both methods.

	Position RMSE [m]		Heading RMSE [deg]	
NMPC+INDI	**0.102**	(↑ 0%)	**5.967**	(↑ 0%)
NMPC+INDI (w/o drag model)	0.249	(↑ 144%)	6.116	(↑ 3%)
NMPC	0.473	(↑ 364%)	12.848	(↑ 115%)
DFBC+INDI	**0.122**	(↑ 0%)	**7.739**	(↑ 0%)
DFBC+INDI (w/o drag model)	0.271	(↑ 122%)	7.745	(↑ 0.07%)
DFBC	0.982	(↑ 705%)	45.845	(↑ 492%)

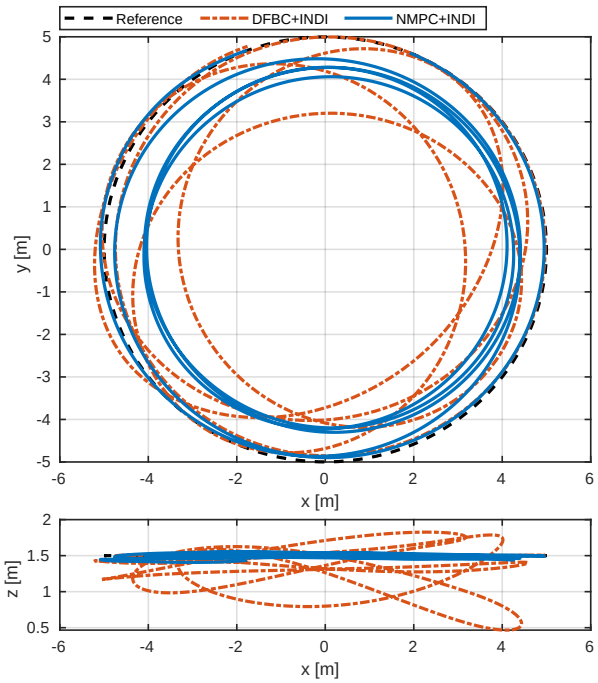


Fig. 14: Top and side view of the trajectory tracking a dynamically infeasible trajectory (black-dash) using NMPC (blue-solid) and DFBC method (red-dash-dot).

C. Tracking Dynamically Infeasible Trajectories

In these real-world experiments, we also track three dynamically infeasible trajectories that exceed the maximum thrust of the tested quadrotor. Comparative results of NMPC and DFBC are given in Table VII. Clearly, NMPC shows significantly higher tracking accuracy than DFBC when tracking these dynamically infeasible trajectories, which is consistent with the simulation results in Sec. V.

Fig. 14 presents the 3D path tracking the *Loop C* reference trajectory. Tracking such a reference loop trajectory requires a higher collective thrust than the maximum thrust of the tested quadrotor. Hence the quadrotor cannot follow the reference trajectory and both control methods make the quadrotor take the shortcut to fly inside the reference trajectory. While DFBC results in a chaotic trajectory, NMPC makes the drone fly a much more regular loop trajectory with a smaller radius than the reference, thus requiring less collective thrust. The difference on the side-view is more distinctive: NMPC has much higher altitude tracking accuracy than the DFBC method.

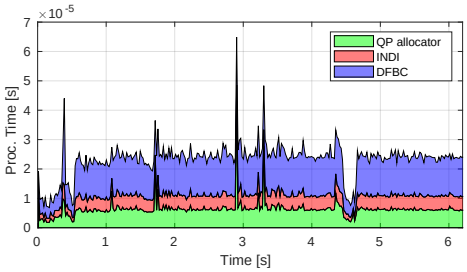


Fig. 15: CPU time of DFBC+INDI while tracking the *Race Track C* trajectory.

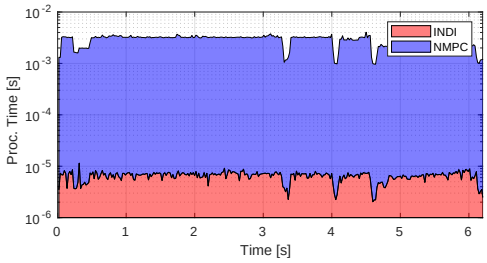


Fig. 16: CPU time of NMPC+INDI while tracking the *Race Track C* trajectory. Note that the vertical axis is presented in the log scale.

D. Computational Time

Fig. 15 and 16 respectively show the CPU time of DFBC and NMPC with INDI as the inner-loop controller while tracking the *Race Track C* trajectory. The computation time of DFBC is generally faster than 0.025 ms, which is significantly faster compared with NMPC that takes around 3 ms. In addition, both QP and INDI modules spend less than 0.01 ms CPU time.

Fig. 17 compares the average computational time of generating each control commands of NMPC and DFBC, with INDI using all the flight data. Each data point in the box plot represents the average computing time of a single flight. As is expected, NMPC requires significantly longer time to generate a single control command (2.7 ms) compared with DFBC (0.020 ms). Be that as it may, both methods can run onboard at sufficiently high frequency ($\geq$100 Hz) to achieve accurate tracking of agile trajectories.

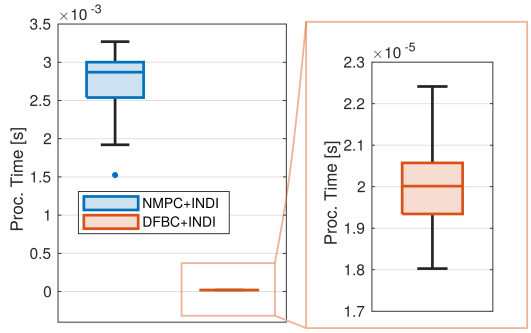


Fig. 17: Processing time to generate each control command of NMPC and DFBC methods.

VII. Discussion

According to the simulation and real-world flight results, we conclude that both NMPC and DFBC, with an INDI inner-loop controller, can track highly aggressive trajectories with similar accuracy as long as the reference trajectories are dynamically *feasible*.

The advantage of NMPC appears when tracking dynamically *infeasible* trajectories that violate the rotor thrust constraints. Even though DFBC also uses the constrained-quadratic programming for control allocation, it only considers a single reference point; thus, the actions taken are too short-sighted to avoid future violations of these constraints. By contrast, NMPC tracks the *infeasible* trajectory using future predictions, including multiple reference points that minimize the tracking error and respect single rotor constraints. Such difference helps NMPC outperform DFBC by 57% and 64% on position and heading accuracy, respectively, if the reference trajectories are dynamically *infeasible*.

NMPC also demonstrates higher robustness against the rotational model mismatch. In real-world experiments, we also performed tests without INDI low-level controller for comparisons. As such, both methods suffered from model uncertainties on the rotational dynamics. In this condition, NMPC was still able to track all the trajectories, whereas DFBC experienced several crashes and had much higher tracking RMSE against NMPC.

However, NMPC also has limitations. For example, the biggest disadvantage of NMPC is that it requires significantly higher computational resources. On our tested hardware, the average solving time of the nonlinear NMPC is around 2.7 ms, while DFBC needs only 0.020 ms, which is around 100 times faster. This renders it impractical to run NMPC on some miniature aerial vehicles with a limited computational budget, such as Crazyflie [43] . By contrast, we can deploy DFBC on light-weighted drones with low-end processors and conduct agile flights at speeds over 20 m/s, as long as the platform has enough thrust-to-weight ratio.

Another disadvantage of NMPC is that it potentially suffers from numerical convergence issues. Unlike DFBC, which has proof of stability or convergence of each sub-module, the nonlinear NMPC used in this comparison relies on the numerical convergence of the nonlinear optimization algorithm. Unfortunately, rigorous proof of its convergence is still an open question. In fact, nonlinear NMPC tends to fail in converging when the current position is too far from the reference, either caused by large external force disturbances or an error in the thrust-to-weight ratio model. For example, our robustness study shows a 10% higher crash rate of NMPC than DFBC when the real mass is 30% higher than the model. In addition, we also observe that NMPC is more prone to fail in converging than DFBC in the presence of large system latency.

In summary, NMPC is not superior to DFBC in all scenarios. If the reference trajectories are dynamically *feasible*, DFBC can achieve similar tracking performance while consuming only 1 percent of the control resource that NMPC requires. However, it is difficult to justify the feasibility of a trajectory near the physical limitations of the platform because

of the model uncertainties such as aerodynamic effects. Hence, NMPC excels at exploiting the full capability of an autonomous drone in a safer and more efficient manner, due to its advantage in handling the infeasibility of reference trajectories. Future work may further hybridize NMPC with differential flatness property to reduce the NMPC computational cost while retaining its advantage in handling dynamically infeasible trajectories. The reason why NMPC failed in converging also needs further investigation in order to deal with larger amount of model uncertainties.

VIII. Conclusion

This work systematically compares NMPC and DFBC, two state-of-the-art quadrotor controllers for tracking agile trajectories. Simulation and real-world experiments are extensively performed to evaluate the performance of both methods in terms of accuracy, robustness, and computational efficiency. We report the advantages and disadvantages of both methods according to the results. This work also evaluates the effect of INDI inner-loop control on both methods. Real-world flight results at up to 20 $\mathrm{m/s}$ demonstrate the necessity of applying an INDI inner-loop on both NMPC and DFBC approaches. The effect of the high-speed aerodynamic drag model is also evaluated, which is found less influential compared to the inner-loop controller.

Acknowledgment

We thank Thomas Längle and Leonard Bauersfeld for their help in experiments and photo rendering. We also appreciate the valuable feedback from Yunlong Song, Dr. Antonio Loquercio, Drew Hanover, and Dr. Xuerui Wang from TU Delft.

References

[1] G. Loianno and D. Scaramuzza, "Special issue on future challenges and opportunities in vision-based drone navigation," 2020.

[2] S. Rajendran and S. Srinivas, "Air taxi service for urban mobility: A critical review of recent developments, future challenges, and opportunities," *Transportation Research Part E: Logistics and Transportation Review*, vol. 143, p. 102090, Nov 2020. arXiv: 2103.01768.

[3] D. Bicego, J. Mazzetto, R. Carli, M. Farina, and A. Franchi, "Nonlinear model predictive control with enhanced actuator model for multi-rotor aerial vehicles with generic designs," *Journal of Intelligent & Robotic Systems*, vol. 100, no. 3, pp. 1213–1247, 2020.

[4] P. Foehn, A. Romero, and D. Scaramuzza, "Time-optimal planning for quadrotor waypoint flight," *Science Robotics*, vol. 6, no. 56, 2021.

[5] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, "Data-driven mpc for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3769–3776, 2021.

[6] M. Kamel, K. Alexis, M. Achtelik, and R. Siegwart, "Fast nonlinear model predictive control for multicopter attitude tracking on so (3)," in *2015 IEEE Conference on Control Applications (CCA)*, pp. 1160–1166, IEEE, 2015.

[7] M. Kamel, T. Stastny, K. Alexis, and R. Siegwart, "Model predictive control for trajectory tracking of unmanned aerial vehicles using robot operating system," in *Robot operating system (ROS)*, pp. 3–39, Springer, 2017.

[8] A. Murilo and R. V. Lopes, "Unified nmpc framework for attitude and position control for a vtol uav," *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, vol. 233, no. 7, pp. 889–903, 2019.

[9] E. Small, P. Sopasakis, E. Fresk, P. Patrinos, and G. Nikolakopoulos, "Aerial navigation in obstructed environments with embedded nonlinear model predictive control," in *2019 18th European Control Conference (ECC)*, pp. 3556–3563, IEEE, 2019.

[10] H. Nguyen, M. Kamel, K. Alexis, and R. Siegwart, "Model predictive control for micro aerial vehicles: A survey," *arXiv preprint arXiv:2011.11104*, 2020.

[11] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 620–626, 2017.

[12] E. Tal and S. Karaman, "Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness," *IEEE Transactions on Control Systems Technology*, 2020.

[13] T. A. Johansen and T. I. Fossen, "Control allocation—a survey," *Automatica*, vol. 49, no. 5, pp. 1087–1103, 2013.

[14] E. J. Smeur, Q. Chu, and G. C. de Croon, "Adaptive incremental nonlinear dynamic inversion for attitude control of micro air vehicles," *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 3, pp. 450–461, 2016.

[15] S. Sun, X. Wang, Q. Chu, and C. de Visser, "Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors," *IEEE Transactions on Robotics*, vol. 37, no. 1, pp. 116–130, 2020.

[16] T. P. Nascimento and M. Saska, "Position and attitude control of multi-rotor aerial vehicles: A survey," *Annual Reviews in Control*, vol. 48, pp. 129–146, 2019.

[17] H. Lee and H. J. Kim, "Trajectory tracking control of multirotors from modelling to experiments: A survey," *International Journal of Control, Automation and Systems*, vol. 15, no. 1, pp. 281–292, 2017.

[18] S. Khatoon, D. Gupta, and L. Das, "Pid & lqr control for a quadrotor: Modeling and simulation," in *2014 international conference on advances in computing, communications and informatics (ICACCI)*, pp. 796–802, IEEE, 2014.

[19] W. Dong, G.-Y. Gu, X. Zhu, and H. Ding, "Modeling and control of a quadrotor uav with aerodynamic concepts," in *Proceedings of World Academy of Science, Engineering and Technology*, no. 77, p. 437, World Academy of Science, Engineering and Technology (WASET), 2013.

[20] H. Voos, "Nonlinear control of a quadrotor micro-uav using feedback-linearization," in *2009 IEEE International Conference on Mechatronics*, pp. 1–6, IEEE, 2009.

[21] T. Madani and A. Benallegue, "Backstepping control for a quadrotor helicopter," in *2006 IEEE/RSJ International*

Conference on Intelligent Robots and Systems, pp. 3255–3260, IEEE, 2006.

[22] E. Fresk and G. Nikolakopoulos, "Full quaternion based attitude control for a quadrotor," in *2013 European control conference (ECC)*, pp. 3864–3869, IEEE, 2013.

[23] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor uav on se (3)," in *49th IEEE conference on decision and control (CDC)*, pp. 5420–5425, IEEE, 2010.

[24] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *2011 IEEE international conference on robotics and automation*, pp. 2520–2525, IEEE, 2011.

[25] R. Mahony, V. Kumar, and P. Corke, "Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor," *IEEE Robotics and Automation magazine*, vol. 19, no. 3, pp. 20–32, 2012.

[26] S. Sun, C. C. de Visser, and Q. Chu, "Quadrotor gray-box model identification from high-speed flight data," *Journal of Aircraft*, vol. 56, no. 2, pp. 645–661, 2019.

[27] H. Huang, G. M. Hoffmann, S. L. Waslander, and C. J. Tomlin, "Aerodynamics and control of autonomous quadrotor helicopters in aggressive maneuvering," in *2009 IEEE international conference on robotics and automation*, pp. 3277–3282, IEEE, 2009.

[28] M. Faessler, D. Falanga, and D. Scaramuzza, "Thrust mixing, saturation, and body-rate control for accurate aggressive quadrotor flight," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 476–482, 2016.

[29] E. Smeur, D. Höppener, and C. De Wagter, "Prioritized control allocation for quadrotors subject to saturation," in *International Micro Air Vehicle Conference and Flight Competition*, no. September, pp. 37–43, 2017.

[30] D. Brescianini and R. D'Andrea, "Tilt-prioritized quadrocopter attitude control," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 2, pp. 376–387, 2018.

[31] H. Zaki, M. Unel, and Y. Yildiz, "Trajectory control of a quadrotor using a control allocation approach," in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 533–539, IEEE, 2017.

[32] M. Kamel, M. Burri, and R. Siegwart, "Linear vs nonlinear mpc for trajectory tracking applied to rotary wing micro aerial vehicles," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 3463–3469, 2017.

[33] M. Bangura and R. Mahony, "Real-time model predictive control for quadrotors," *IFAC Proceedings Volumes*, vol. 47, no. 3, pp. 11773–11780, 2014.

[34] K. Alexis, G. Nikolakopoulos, and A. Tzes, "On trajectory tracking model predictive control of an unmanned quadrotor helicopter subject to aerodynamic disturbances," *Asian Journal of Control*, vol. 16, no. 1, pp. 209–224, 2014.

[35] B. Houska, H. J. Ferreau, and M. Diehl, "Acado toolkit—an open-source framework for automatic control and dynamic optimization," *Optimal Control Applications and Methods*, vol. 32, no. 3, pp. 298–312, 2011.

[36] Y. Chen, M. Bruschetta, E. Picotti, and A. Beghi, "Matmpc-a matlab based toolbox for real-time nonlinear model predictive control," in *2019 18th European Control Conference (ECC)*, pp. 3365–3370, IEEE, 2019.

[37] J. A. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "Casadi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[38] M. Jacquet and A. Franchi, "Motor and perception constrained nmpc for torque-controlled generic aerial vehicles," *IEEE Robotics and Automation Letters*, 2020.

[39] M. Diehl, H. G. Bock, H. Diedam, and P.-B. Wieber, "Fast direct multiple shooting algorithms for optimal robot control," in *Fast motions in biomechanics and robotics*, pp. 65–93, Springer, 2006.

[40] R. Verschueren, G. Frison, D. Kouzoupis, N. van Duijkeren, A. Zanelli, R. Quirynen, and M. Diehl, "Towards a modular software package for embedded optimization," *IFAC-PapersOnLine*, vol. 51, no. 20, pp. 374–380, 2018.

[41] H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, "qpoases: A parametric active-set algorithm for quadratic programming," *Mathematical Programming Computation*, vol. 6, no. 4, pp. 327–363, 2014.

[42] X. Wang, E.-J. Van Kampen, Q. Chu, and P. Lu, "Stability analysis for incremental nonlinear dynamic inversion control," *Journal of Guidance, Control, and Dynamics*, vol. 42, no. 5, pp. 1116–1129, 2019.

[43] W. Giernacki, M. Skwierczyński, W. Witwicki, P. Wroński, and P. Kozierski, "Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering," in *2017 22nd International Conference on Methods and Models in Automation and Robotics (MMAR)*, pp. 37–42, IEEE, 2017.